

OSSP (Operating Systems And Systems Programming) - 25CS2104E

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Code : -

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    char cmd[100];
    printf("Enter Linux command: ");
    scanf("%s", cmd);
    pid_t pid = fork();
    if(pid == 0)
    {
        printf("Child PID : %d\n", getpid());
        execlp(cmd, cmd, NULL);
        perror("Execution Failed");
    }
    else if(pid > 0)
    {
        printf("Parent PID : %d\n", getpid());
        wait(NULL);
        printf("Child process completed.\n");
    }
    else
    {
        printf("Fork Failed\n");
    }
    return 0;
}
```

Output:-

```
shashank@LAPTOP-24KPROR8:~$ nano practical1.c
shashank@LAPTOP-24KPROR8:~$ gcc practical1.c -o practical1
shashank@LAPTOP-24KPROR8:~$ ./practical1
```

Enter Linux command: ls

Parent PID : 602

Child PID : 603

Copy	Display	Exec	Fork	OS.txt	Parentchild	Prac1	User	W
ait	Wait2	firstpg.txt	hello.c		practical1.c			
Copy.c	Display.c	Exec.c	Fork.c	OSLab	Parentchild.c	Prac1.c	User.c	W
ait.c	Wait2.c	hello		practical1	samplePg.c			

Child process completed.

```
shashank@LAPTOP-24KPROR8:~$ uname -a
```

Linux LAPTOP-24KPROR8 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC
IC Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux

```
shashank@LAPTOP-24KPROR8:~$ lscpu
```

Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 39 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 12
On-line CPU(s) list: 0-11
Vendor ID: GenuineIntel
Model name: 13th Gen Intel(R) Core(TM) i5-13420H
CPU family: 6
Model: 186
Thread(s) per core: 2
Core(s) per socket: 6
Socket(s): 1
Stepping: 2
BogoMIPS: 5222.39
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtr

```
shashank@LAPTOP-24KPROR8:~$ lsblk
```

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINTS
sda	8:0	0	356.9M	1	disk	
sdb	8:16	0	159.4M	1	disk	
sdc	8:32	0	2G	0	disk	[SWAP]
sdd	8:48	0	1T	0	disk	/mnt/wslg/distro

```
shashank@LAPTOP-24KPROR8:~$ ps -f
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
shashank	397	396	0	13:20	pts/0	00:00:00	-bash
shashank	707	397	0	13:32	pts/0	00:00:00	ps -f

```
shashank@LAPTOP-24KPROR8:~$ top
top - 13:33:21 up 13 min, 1 user, load average: 0.00, 0.02, 0.00
Tasks: 24 total, 1 running, 23 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7772.6 total, 7225.0 free, 531.4 used, 167.0 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 7241.2 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	24156	15396	11572	S	0.0	0.2	0:00.65	systemd
2	root	20	0	3180	2208	2072	S	0.0	0.0	0:00.00	init-systemd(Ub
6	root	20	0	3180	2040	1940	S	0.0	0.0	0:00.00	init
46	root	19	-1	50400	16808	15544	S	0.0	0.2	0:00.19	systemd-journal
78	systemd+	20	0	22416	14476	11984	S	0.0	0.2	0:00.08	systemd-resolve
84	root	20	0	35360	12280	9196	S	0.0	0.2	0:00.26	systemd-udev
113	root	20	0	2888	1944	1820	S	0.0	0.0	0:00.05	chronyd-starter
117	root	20	0	4472	3172	2920	S	0.0	0.0	0:00.01	cron
123	message+	20	0	8816	5340	4668	S	0.0	0.1	0:00.06	dbus-daemon
155	root	20	0	44096	29712	14572	S	0.0	0.4	0:00.32	networkd-dispat
163	root	20	0	18440	9452	8204	S	0.0	0.1	0:00.09	systemd-logind
205	syslog	20	0	220548	5876	4616	S	0.0	0.1	0:00.09	rsyslogd
222	root	20	0	5492	2756	2572	S	0.0	0.0	0:00.01	agetty
240	root	20	0	123460	32728	18120	S	0.0	0.4	0:00.27	unattended-upgr
248	_chrony	20	0	20424	6348	5576	S	0.0	0.1	0:00.05	chronyd
262	_chrony	20	0	12096	2428	1788	S	0.0	0.0	0:00.00	chronyd
395	root	20	0	3184	1112	980	S	0.0	0.0	0:00.00	SessionLeader
396	root	20	0	3200	1248	1108	S	0.0	0.0	0:00.03	Relay(397)

2. Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

Code:-

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
    int src, dest;
    char buffer[1024];
    int bytes;

    src=open("somefile.txt", O_RDONLY);

    if(src < 0)
    {
        printf("Cannot open source file\n");
        return 1;
    }
    dest=open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    while((bytes = read(src, buffer, sizeof(buffer))) > 0)
    {
        write(dest, buffer, bytes);
    }
    close(src);
    close(dest);

    printf("File copied successfully.\n");
    return 0;
}
```

Output:-

```
shashank@LAPTOP-24KPR0R8:~$ nano Prac2.c
shashank@LAPTOP-24KPR0R8:~$ gcc Prac2.c -o Prac2
shashank@LAPTOP-24KPR0R8:~$ ./Prac2
File copied successfully.
shashank@LAPTOP-24KPR0R8:~$ cat somefile.txt
Hello, this is a test file.
Line 2 of the file.
Line 3 of the file.
```

